

CARRERA: Maestría de Software

ASIGNATURA: Patrones de Diseño de APIs

NRO. PRÁCTICA:

04

TÍTULO PRÁCTICA: GraphQL

OBJETIVO : Optimizar de Modelos de Datos en Proveedores de Servicios con GraphQL

INSTRUCCIONES

1. Diseñar e implementar un esquema GraphQL
2. Identificar el problema de rendimiento Consulta N+1
3. Mitigar el problema utilizando el patrón arquitectónico DataLoader

1. Contexto del Problema y Objetivos de Negocio

Para conectar con la **cadena de valor** de la empresa ficticia "*LogiTech SaaS*", el equipo de Frontend requiere desplegar un tablero unificado de clientes con sus respectivas facturas históricas y consumos de infraestructura.

Actualmente, una solución basada en **REST** requiere que el cliente realice 1 petición para obtener la lista de N clientes, y luego N peticiones individuales para recuperar las facturas de cada uno (`/clientes` y luego `/clientes/{id}/facturas`). Esto genera un problema crítico de rendimiento conocido como **Under-fetching** y satura los canales de red móviles.

Objetivos del Laboratorio:

Diseñar e implementar un esquema GraphQL (*Schema-First*) que unifique el modelo de datos en un solo grafo.

Identificar el problema de rendimiento **Consulta N+1** mediante logs de consola en la base de datos simulada.

Mitigar el problema utilizando el patrón arquitectónico **DataLoader** (agrupamiento por lotes y almacenamiento en caché por petición).

2. Requisitos Previos e Infraestructura

Entorno: Node.js (v18 o superior) instalado.

Librerías Core: graphql, @apollo/server (o la alternativa correspondiente en el lenguaje elegido), y dataloader.

Código Base Proporcionado por el Docente: Un repositorio inicial con una base de datos simulada en memoria (Mock DB) que expone funciones asíncronas para buscar clientes y facturas, las cuales incluyen un `console.log("[DB QUERY] Ejecutando búsqueda...")` cada vez que son invocadas.

3. Entregables Técnicos

Los maestrantes deberán subir al repositorio (GitHub/GitLab) el código estructurado que contenga:

schema.graphql: Definición estricta del Sistema de Tipos (SDL) incluyendo los tipos `Cliente`, `Factura`, la query de entrada y las relaciones jerárquicas.

resolvers.js: Implementación de los resolutores. Debe presentarse en dos ramas o archivos separados: uno con la solución ingenua (que provoca el error N+1) y otro con la solución optimizada.

dataloaders.js: Implementación de la función de carga por lotes (*Batch Function*) para resolver las facturas de múltiples clientes en un solo viaje de datos.

Código Base

<https://github.com/pmpvhr/graphql.git>

Docente / Técnico Docente:

Firma: _____

Resolución CS N° 076-04-2016-04-20